

3.2 神经网络优化

梯度下降、反向传播与训练技巧

Lecture Notes · 依据康奇宇老师课件整理

✔ 一页读懂本讲

训练神经网络就是求解 $\theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$ 。本讲依次回答三个问题：

1. **往哪里走？——梯度下降**：沿负梯度方向迭代；数据太大时，用一个 mini-batch 上的梯度代替全量梯度，即**随机梯度下降**（SGD）。
2. **梯度怎么算？——反向传播**：神经网络是复合函数，用链式法则求导；从输出层向输入层逐层传递“误差项”，中间结果存储复用。核心公式只有三条。
3. **怎样走得又快又稳、还不过拟合？——训练技巧**：样本打乱、批量大小、学习率及其衰减（管优化）；正则化、提前终止、Dropout（管泛化）；以及参数初始化（管起点）。

用框架写代码时，这三件事分别对应：`optimizer.step()`、`loss.backward()`，以及围绕它们的各种超参数。

目录

1 梯度下降与随机梯度下降	2
1.1 前馈神经网络与记号	2
1.2 梯度下降	3
1.3 随机梯度下降（SGD）	4
1.4 随机梯度下降 vs. 梯度下降	5
1.5 终止条件	5
2 反向传播算法	6
2.1 问题：如何高效计算梯度	6
2.2 单隐层网络上的推导	6
2.3 一个数值例子	8
2.4 一般情形：向量形式	8
2.5 训练流程小结	9
3 深度学习的三个步骤与框架入门	10
3.1 Keras	10
3.2 PyTorch	10

4 随机梯度下降可能存在的问题	11
4.1 问题一：所有参数共用一个学习率	11
4.2 问题二：梯度很小的地方	12
4.3 改进的优化算法（补充）	12
5 神经网络训练优化要点与技巧	14
5.1 样本随机性	14
5.2 批量大小	14
5.3 学习率与学习率衰减	15
5.4 正则化	16
5.5 提前终止（Early Stop）	17
5.6 Dropout	17
6 参数初始化	20
6.1 参数不能全部初始化为 0	20
6.2 初始化方法	20
6.3 基于固定方差的随机初始化	21
7 课后习题与参考解答	21
8 课件勘误与补充说明	22
9 自测问题	22

1 梯度下降与随机梯度下降

1.1 前馈神经网络与记号

前馈神经网络（多层感知机, MLP）〔PPT 第 4 页〕：各神经元分别属于不同的层，层内无连接；相邻两层之间的神经元全部两两连接（**全连接**）；整个网络中无反馈，信号从输入层向输出层单向传播。

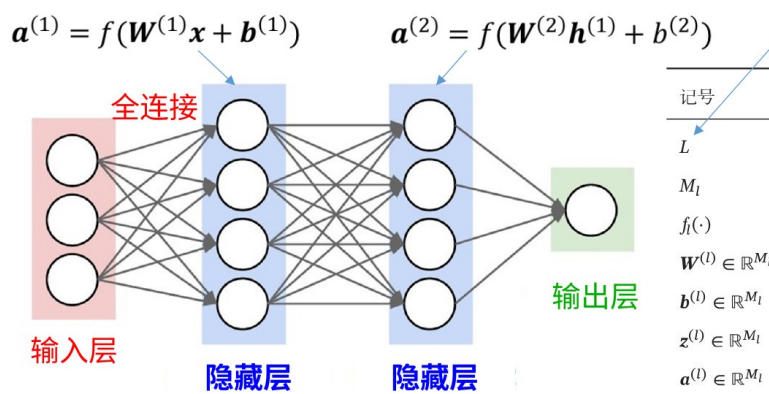


图 1 前馈神经网络。统计层数 L 时一般只考虑隐藏层和输出层。

记号	含义	记号	含义
L	神经网络的层数	$\mathbf{W}^{(l)} \in \mathbb{R}^{M_l \times M_{l-1}}$	第 $l-1$ 层到第 l 层的权重矩阵
M_l	第 l 层神经元的个数	$\mathbf{b}^{(l)} \in \mathbb{R}^{M_l}$	第 $l-1$ 层到第 l 层的偏置
$f_l(\cdot)$	第 l 层的激活函数	$\mathbf{z}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层的净输入（净活性值）
		$\mathbf{a}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层的输出（活性值）

逐层计算： $\mathbf{a}^{(0)} = \mathbf{x}$, $\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$, $\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)})$ 。

1.2 梯度下降

给定训练集 $D = \{(\mathbf{x}^{(n)}, \mathbf{y}^{(n)})\}_{n=1}^N$ ，将每个样本输入网络得到输出 $\hat{\mathbf{y}}^{(n)}$ ，在数据集 D 上的**结构化风险函数**为 [PPT 第 5 页]

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)}) + \frac{1}{2} \lambda \|\mathbf{W}\|_F^2, \quad (1.1)$$

第一项是经验风险（平均损失），第二项是正则化项（见第 5.4 小节）。训练即求 $\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$ 。

梯度下降（Gradient Descent, GD）

(1) 初始化 $\boldsymbol{\theta}^0$ ；(2) 重复：计算梯度 $\nabla \mathcal{L}(\boldsymbol{\theta}^{t-1})$ ，更新参数

$$\boldsymbol{\theta}^t \leftarrow \boldsymbol{\theta}^{t-1} - \alpha \nabla \mathcal{L}(\boldsymbol{\theta}^{t-1}), \quad \alpha > 0 \text{ 为学习率。} \quad (1.2)$$

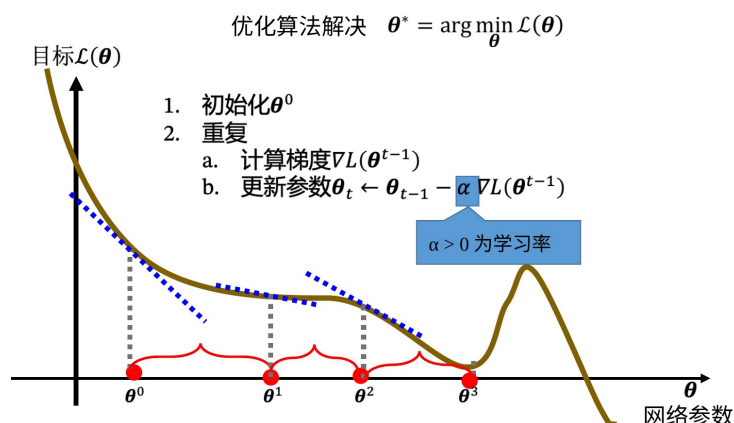


图 2 梯度下降：每一步沿当前点切线下降的方向移动，步长正比于梯度大小。〔PPT 第 6 页〕

直观理解：梯度是函数值上升最快的方向，负梯度就是下降最快的方向；坡越陡，梯度越大，步子越大；到了平坦处，梯度趋于 0，步子自动变小。

💡 思考题：为什么梯度下降能保证损失非递增？有什么前提条件？

由一阶泰勒展开， $\mathcal{L}(\theta - \alpha g) \approx \mathcal{L}(\theta) - \alpha \|g\|^2$ （其中 $g = \nabla \mathcal{L}(\theta)$ ），只要 α 足够小，损失就会下降。“足够小”可以精确化：若 $\nabla \mathcal{L}$ 是 β -Lipschitz 连续的（ $\mathcal{L}$ 是 β -光滑的），则

$$\mathcal{L}(\theta - \alpha g) \leq \mathcal{L}(\theta) - \alpha \|g\|^2 + \frac{\beta \alpha^2}{2} \|g\|^2 = \mathcal{L}(\theta) - \alpha \left(1 - \frac{\alpha \beta}{2}\right) \|g\|^2.$$

当 $\alpha \leq 1/\beta$ 时， $\mathcal{L}(\theta^t) \leq \mathcal{L}(\theta^{t-1}) - \frac{\alpha}{2} \|g\|^2$ ，损失单调不增。**前提条件：**（1）损失函数可微且梯度变化不太剧烈（光滑）；（2）学习率足够小；（3）使用的是**精确的全量梯度**——随机梯度下降并不保证每一步都下降。另外，“非递增”只保证收敛到梯度为 0 的点，不保证是全局最优。

1.3 随机梯度下降 (SGD)

数据集可能很大，无法全部放入内存来计算梯度。一般采用**小批量随机梯度下降法**：每次从数据集中采样一部分样本（称为一个 **batch**，大小为 B ），计算 batch 上的梯度并更新参数。〔PPT 第 7 页〕

	梯度下降	小批量随机梯度下降
目标	$\mathcal{L}_D = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)}) + \frac{1}{2} \lambda \ \mathbf{W}\ _F^2$	$\mathcal{L}_B = \frac{1}{B} \sum_{i=1}^B \mathcal{L}(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)}) + \frac{1}{2} \lambda \ \mathbf{W}\ _F^2$
数据项梯度	$\frac{1}{N} \sum_{n=1}^N \frac{\partial \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)})}{\partial \mathbf{W}^{(l)}}$	$\frac{1}{B} \sum_{i=1}^B \frac{\partial \mathcal{L}(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)})}{\partial \mathbf{W}^{(l)}}$

为什么可行？ 若 batch 是从 D 中均匀随机采样的，则 $\mathbb{E}[\nabla \mathcal{L}_B] = \nabla \mathcal{L}_D$ ：小批量梯度是全量梯度的**无偏估计**，只是带有噪声，方差大约与 $1/B$ 成正比（见第 5.2 小节）。

💡 思考题：数据集规模、模型大小与 GPU 内存之间的关系

训练时显存主要由两部分构成：

- **与模型大小成正比**：参数本身、参数的梯度、优化器状态（如动量、Adam 的一阶和二阶矩）。使用 Adam 时，这部分大约是参数数量的 4 倍。
- **与批量大小成正比**：前向传播中各层的激活值必须保存下来供反向传播使用，大小 $\approx B \times$ （每个样本的激活总量）。

采用小批量之后，**数据集规模 N 不再影响显存**，只影响一个 epoch 需要的迭代次数 N/B 。所以：模型越大，留给激活的显存越少， B 就只能越小。

1.4 随机梯度下降 vs. 梯度下降

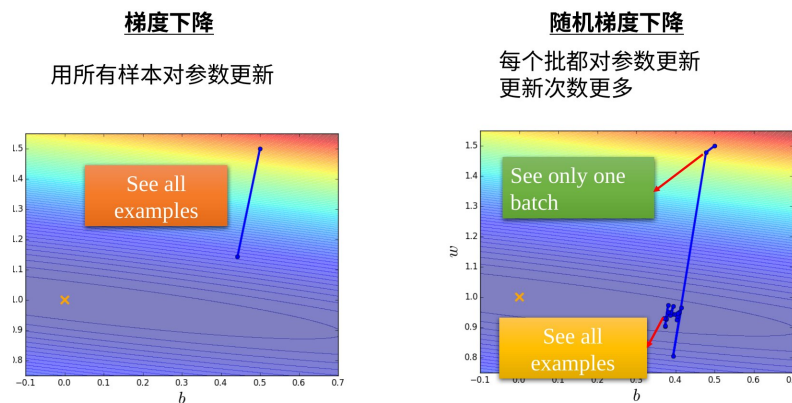


图 3 看完全部样本之后：梯度下降只更新了一次；随机梯度下降每个 batch 更新一次，轨迹有噪声，但已经走了很远。〔PPT 第 8 页〕

两个术语：一次 **iteration**（迭代）指用一个 batch 更新一次参数；一个 **epoch**（回合）指把整个训练集完整过一遍，包含 N/B 次迭代。

1.5 终止条件

算法 2.1: 随机梯度下降法

输入：训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$ ，验证集 $\mathcal{V}$ ，学习率 α

```

1 随机初始化  $\theta$ ;
2 repeat
3   对训练集  $\mathcal{D}$  中的样本随机重排序;
4   for  $n = 1 \cdots N$  do
5     从训练集  $\mathcal{D}$  中选取样本  $(\mathbf{x}^{(n)}, y^{(n)})$ ;
6     // 更新参数
7      $\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}(\theta; \mathbf{x}^{(n)}, y^{(n)})}{\partial \theta}$ ;
8   end
9 until 模型  $f(\mathbf{x}; \theta)$  在验证集  $\mathcal{V}$  上的错误率不再下降;
输出:  $\theta$ 
```

图 4 随机梯度下降算法（图中为每次取单个样本的情形，即 $B = 1$ ）。〔PPT 第 9 页〕

要点：(1) 每个 epoch 开始前对训练集随机重排；(2) 终止条件不是“训练损失不再下降”，而是**模型在验证集 $\mathcal{V}$ 上的错误率不再下降**——这其实就是第 5.5 小节的“提前终止”。

2 反向传播算法

2.1 问题：如何高效计算梯度

神经网络是一个复杂的**复合函数** [PPT 第 11 页]，例如 $y = f^4(f^3(f^2(f^1(\mathbf{x}, \boldsymbol{\theta}^{(1)}), \boldsymbol{\theta}^{(2)}), \boldsymbol{\theta}^{(3)}), \boldsymbol{\theta}^{(4)})$ 。由链式求导法则：

$$\frac{\partial y}{\partial \boldsymbol{\theta}^{(4)}} = \frac{\partial f^4}{\partial \boldsymbol{\theta}^{(4)}}, \quad \frac{\partial y}{\partial \boldsymbol{\theta}^{(3)}} = \frac{\partial f^3}{\partial \boldsymbol{\theta}^{(3)}} \frac{\partial y}{\partial f^3}, \quad \frac{\partial y}{\partial \boldsymbol{\theta}^{(2)}} = \frac{\partial f^2}{\partial \boldsymbol{\theta}^{(2)}} \frac{\partial f^3}{\partial f^2} \frac{\partial y}{\partial f^3}, \quad \frac{\partial y}{\partial \boldsymbol{\theta}^{(1)}} = \frac{\partial f^1}{\partial \boldsymbol{\theta}^{(1)}} \frac{\partial f^2}{\partial f^1} \frac{\partial f^3}{\partial f^2} \frac{\partial y}{\partial f^3}.$$

带颜色的因子 $\partial f^l / \partial f^{l-1}$ 在计算不同层参数的导数时**重复出现**，可以**存储复用**。

再看一个三层的具体例子 [PPT 第 12 页]： $y = \sigma(\mathbf{W}_3 \sigma(\mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x})))$ ，记 $\mathbf{z}_i = \mathbf{W}_i \mathbf{a}_{i-1}$ ， $\mathbf{a}_i = \sigma(\mathbf{z}_i)$ ，则

$$\frac{\partial y}{\partial \mathbf{W}_3} = \frac{\partial y}{\partial \mathbf{z}_3} \frac{\partial \mathbf{z}_3}{\partial \mathbf{W}_3}, \quad \frac{\partial y}{\partial \mathbf{W}_2} = \frac{\partial y}{\partial \mathbf{z}_3} \frac{\partial \mathbf{z}_3}{\partial \mathbf{a}_2} \frac{\partial \mathbf{a}_2}{\partial \mathbf{z}_2} \frac{\partial \mathbf{z}_2}{\partial \mathbf{W}_2}, \quad \frac{\partial y}{\partial \mathbf{W}_1} = \frac{\partial y}{\partial \mathbf{z}_3} \frac{\partial \mathbf{z}_3}{\partial \mathbf{a}_2} \frac{\partial \mathbf{a}_2}{\partial \mathbf{z}_2} \frac{\partial \mathbf{a}_1}{\partial \mathbf{z}_1} \frac{\partial \mathbf{z}_1}{\partial \mathbf{W}_1}.$$

✔ 两个观察 $\Rightarrow$ 反向传播的两个阶段

- **观察 1**：计算梯度需要用到各层的激活值（如 $\partial \mathbf{z}_2 / \partial \mathbf{W}_2$ 依赖 $\mathbf{a}_1$ ） $\Rightarrow$ 先做一次**前向传播**，并把激活值存下来。
- **观察 2**：部分中间结果（如 $\frac{\partial y}{\partial \mathbf{z}_3}$ 、 $\frac{\partial \mathbf{a}_2}{\partial \mathbf{z}_2}$ ）可以复用 $\Rightarrow$ 从最深层开始，向浅层依次计算，即**后向传播**。

反向传播算法就是根据前馈网络的特点设计的高效求导方法。更一般的计算方法是**自动微分** (Automatic Differentiation, AD)，深度学习框架中的 `backward()` 即是它的实现。

2.2 单隐层网络上的推导

设定 [PPT 第 13 页]：输入 $\mathbf{x} \in \mathbb{R}^d$ ，隐层 q 个神经元，输出 $\mathbf{y} \in \mathbb{R}^l$ ，标记为 $\mathbf{d}$ 。

- 第 h 个隐层神经元：输入 $z_h^{(1)} = \sum_{i=1}^d w_{ih}^{(1)} x_i + b_h^{(1)}$ ，输出 $a_h = f_1(z_h^{(1)})$ ；
- 第 j 个输出神经元：输入 $z_j^{(2)} = \sum_{h=1}^q w_{hj}^{(2)} a_h + b_j^{(2)}$ ，输出 $y_j = f_2(z_j^{(2)})$ ；
- 需要优化的参数个数： $dq + q + ql + l = (d + l + 1)q + l$ ；
- 损失函数取平方损失 $\mathcal{L}(\mathbf{d}, \mathbf{y}) = \frac{1}{2} \sum_{j=1}^l (y_j - d_j)^2$ 。

BP 是一个迭代学习算法，每一轮对任意参数 v 按 $v \leftarrow v + \Delta v$ 更新，其中 $\Delta v = -\eta \partial \mathcal{L} / \partial v$ ， η 为学习率。

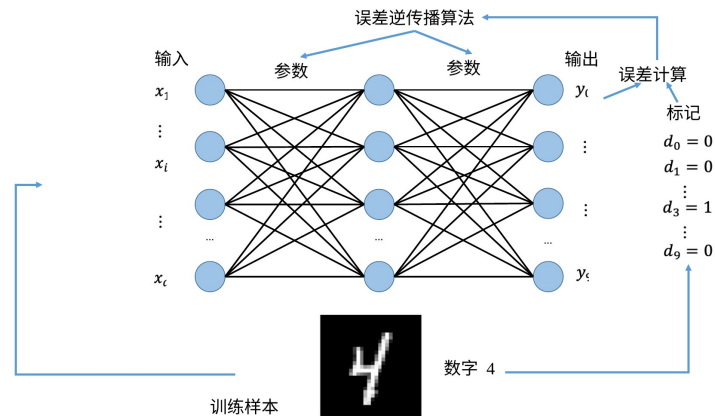


图5 误差逆传播的整体流程：训练样本前向得到输出，与标记比较计算误差，再把误差逆向传回去调整各层参数。〔PPT 第 14 页〕

隐层到输出层的权重〔PPT 第 16 页〕。 $w_{hj}^{(2)}$ 先影响 $z_j^{(2)}$ ，再影响 y_j ，最后影响损失：

$$\Delta w_{hj}^{(2)} = -\eta \frac{\partial \mathcal{L}}{\partial w_{hj}^{(2)}} = -\eta \frac{\partial \mathcal{L}}{\partial y_j} \cdot \frac{\partial y_j}{\partial z_j^{(2)}} \cdot \frac{\partial z_j^{(2)}}{\partial w_{hj}^{(2)}} = \eta \underbrace{(d_j - y_j) f_2'(z_j^{(2)})}_{e_j^{(2)}} a_h = \eta e_j^{(2)} a_h. \quad (2.1)$$

输入层到隐层的权重〔PPT 第 17 页〕。 a_h 会影响**所有**输出神经元，所以要对 j 求和：

$$\frac{\partial \mathcal{L}}{\partial a_h} = \sum_j \frac{\partial \mathcal{L}}{\partial z_j^{(2)}} \frac{\partial z_j^{(2)}}{\partial a_h} = -\sum_j e_j^{(2)} w_{hj}^{(2)}, \quad \Delta w_{ih}^{(1)} = \eta f_1'(z_h^{(1)}) \underbrace{\sum_j e_j^{(2)} w_{hj}^{(2)}}_{e_h^{(1)}} x_i = \eta e_h^{(1)} x_i. \quad (2.2)$$

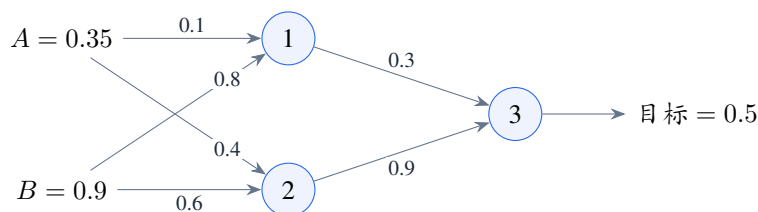
✔ 统一的形式

$$\Delta w = \eta \times (\text{这条边终点神经元的误差项}) \times (\text{这条边起点神经元的输出}).$$

输出层的误差项由预测误差直接得到；隐层的误差项由**后一层的误差项按连接权重加权求和**，再乘以本层激活函数的导数得到——误差就是这样“逆向传播”的。

2.3 一个数值例子

激活函数为 Sigmoid ($\sigma'(z) = o(1-o)$, o 为输出), 学习率 $\eta = 1$ 。 [PPT 第 18 页]



步骤	计算
前向: 神经元 1	输入 $0.35 \times 0.1 + 0.9 \times 0.8 = 0.755$, 输出 $o_1 = \sigma(0.755) = 0.68$
前向: 神经元 2	输入 $0.35 \times 0.4 + 0.9 \times 0.6 = 0.68$, 输出 $o_2 = \sigma(0.68) = 0.6637$
前向: 神经元 3	输入 $0.3 \times 0.68 + 0.9 \times 0.6637 = 0.80133$, 输出 $o = 0.69$, 误差 $t - o = -0.19$
后向: 输出层误差项	$g = (t - o) o(1 - o) = (0.5 - 0.69) \times 0.69 \times (1 - 0.69) = -0.0406$
后向: 隐层误差项	$e_1 = g w_{13} o_1(1 - o_1) = -0.0406 \times 0.3 \times 0.68 \times 0.32 = -2.650 \times 10^{-3}$ $e_2 = g w_{23} o_2(1 - o_2) = -0.0406 \times 0.9 \times 0.6637 \times 0.3363 = -8.156 \times 10^{-3}$
更新: 输出层权重	$w_{13}^+ = 0.3 + g o_1 = 0.2724$, $w_{23}^+ = 0.9 + g o_2 = 0.8731$
更新: 隐层权重	$w_{A1}^+ = 0.1 + e_1 A = 0.0991$, $w_{B1}^+ = 0.8 + e_1 B = 0.7976$ $w_{A2}^+ = 0.4 + e_2 A = 0.3971$, $w_{B2}^+ = 0.6 + e_2 B = 0.5927$
验证	用新权重重新前向: $o_1 = 0.6797$, $o_2 = 0.6620$, $o = 0.6820$; 误差绝对值从 0.19 降到 0.1820

⚠ 一个容易犯的错误

计算隐层误差项 e_1, e_2 时, 使用的是**更新之前**的权重 $w_{13} = 0.3$ 、 $w_{23} = 0.9$ 。应当先用旧参数把所有梯度算完, 再统一更新。

2.4 一般情形: 向量形式

定义误差项 [PPT 第 19 页]: $e^{(l)} \triangleq \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$, 即损失对第 l 层净输入的梯度, 它衡量第 l 层神经元对最终损失的“责任”。

(1) **误差项 $\rightarrow$ 参数梯度**。 $\mathbf{z}^{(l)}$ 的第 k 个分量为 $z_k^{(l)} = \langle \mathbf{w}_k^{(l)}, \mathbf{a}^{(l-1)} \rangle + b_k^{(l)}$, 其中 $\mathbf{w}_k^{(l)}$ 是 $\mathbf{W}^{(l)}$ 的第 k 行, 于是 $\frac{\partial z_k^{(l)}}{\partial w_{ij}^{(l)}} = \delta(k=i) a_j^{(l-1)}$: $w_{ij}^{(l)}$ 只影响第 i 个净输入。由链式法则,

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(l)}} = \left\langle \mathbf{e}^{(l)}, \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} \right\rangle = e_i^{(l)} a_j^{(l-1)} \implies \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \mathbf{e}^{(l)} (\mathbf{a}^{(l-1)})^\top; \quad \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} = \mathbf{I} \implies \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \mathbf{e}^{(l)}. \quad (2.3)$$

(2) **误差项的递推**。 [PPT 第 20 页] 由 $\mathbf{z}^{(l+1)} = \mathbf{W}^{(l+1)} \mathbf{a}^{(l)} + \mathbf{b}^{(l+1)}$ 和 $\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)})$,

$$\frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} = \mathbf{W}^{(l+1)}, \quad \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} = \text{diag}(\nabla f_l(\mathbf{z}^{(l)})) \quad (\text{激活函数逐元素作用, 雅可比矩阵是对角阵}),$$

代入链式法则 $e^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l+1)}} \cdot \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} \cdot \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}}$, 得到误差项的递推式。

✔ 后向传播算法的核心公式组

$$\mathbf{e}^{(l)} = \nabla f_l(\mathbf{z}^{(l)}) \odot \left((\mathbf{W}^{(l+1)})^\top \mathbf{e}^{(l+1)} \right), \quad (2.4)$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{W}^{(l)}} = \mathbf{e}^{(l)} (\mathbf{a}^{(l-1)})^\top, \quad (2.5)$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \mathbf{e}^{(l)}. \quad (2.6)$$

观察: $\mathbf{e}^{(l+1)}$ 可以用来计算 $\mathbf{e}^{(l)}$ 。只需从输出层的 $\mathbf{e}^{(L)}$ 出发, 逐层向前递推一遍, 就得到所有参数的梯度。[PPT 第 21 页]

几点说明:

- **记法:** 课件把误差项写成行向量, 故式 (2.4) 记作 $\nabla f_l(\mathbf{z}^{(l)}) \odot (\mathbf{e}^{(l+1)} \mathbf{W}^{(l+1)})$; 本笔记统一用列向量, 含义相同。维度检查: $(\mathbf{W}^{(l+1)})^\top \in \mathbb{R}^{M_l \times M_{l+1}}$, $\mathbf{e}^{(l+1)} \in \mathbb{R}^{M_{l+1}}$, 乘积为 M_l 维, 与 $\mathbf{z}^{(l)}$ 一致; 式 (2.5) 是 $M_l \times M_{l-1}$ 的外积, 与 $\mathbf{W}^{(l)}$ 同形。
- **符号约定:** 上一小节 (单隐层推导与数值例子) 中的 e 定义为负梯度 $-\partial \mathcal{L} / \partial z$, 更新写作 $w \leftarrow w + \eta e a$; 本小节的 $\mathbf{e}^{(l)}$ 定义为梯度 $+\partial \mathcal{L} / \partial \mathbf{z}^{(l)}$, 更新写作 $\mathbf{W} \leftarrow \mathbf{W} - \alpha \mathbf{e} \mathbf{a}^\top$ 。两者相差一个负号, 结果完全一致。
- **起点 $\mathbf{e}^{(L)}$ (补充):** 平方损失 + 激活 f_L 时, $\mathbf{e}^{(L)} = (\hat{\mathbf{y}} - \mathbf{y}) \odot \nabla f_L(\mathbf{z}^{(L)})$; Softmax + 交叉熵损失时, 形式特别简洁: $\mathbf{e}^{(L)} = \hat{\mathbf{y}} - \mathbf{y}$ 。
- **与 3.3 节的联系:** 式 (2.4) 逐层递推, 意味着浅层的误差项是许多个 “ $\nabla f \odot \mathbf{W}^\top$ ” 因子的连乘——这正是下一节 “梯度消失” 问题的来源。

2.5 训练流程小结

前馈神经网络的训练过程可以分为以下步骤 [PPT 第 22 页]:

1. **数据准备:** 从训练集中随机采样 k 个样本, 作为一个训练 batch;
2. **前向计算:** 从第一层开始计算每一层的状态 (净输入) 和激活值, 直到最后一层;
3. **反向计算:** 从最后一层开始计算每一层的误差项和参数的偏导数;
4. **更新参数:** 使用合适的学习率更新每一层的参数。

代价: 一次反向传播的计算量与一次前向传播同阶 (约 2 倍); 但前向的激活值必须全部保存, 这是训练比推理更占显存的原因。

3 深度学习的三个步骤与框架入门



图 6 深度学习的三个步骤：定义网络 → 损失函数 → 优化。〔PPT 第 24 页〕

三个步骤分别确定了：**函数族**（模型能表示什么）、**好坏的标准**（希望模型学成什么样）、**搜索的方法**（怎样找到好的参数）。框架替我们完成了其中最繁琐的部分——自动求导。

3.1 Keras

```

1 model = Sequential()                                # step 1: define the
   network
2 model.add(Dense(output_dim=64, input_dim=100))
3 model.add(Activation("relu"))
4 model.add(Dense(output_dim=10))
5 model.add(Activation("softmax"))
6 model.compile(loss='categorical_crossentropy',      # step 2: loss
   optimizer='sgd', metrics=['accuracy'])           # step 3: optimizer
7
8 model.fit(X_train, Y_train, nb_epoch=5, batch_size=32) # train
9 loss = model.evaluate(X_test, Y_test, batch_size=32) # evaluate
  
```

〔PPT 第 25 页〕注：课件中的代码是 Keras 1.x 的写法；在新版本中，output_dim 已改为 units，nb_epoch 已改为 epochs。

3.2 PyTorch

```

1 model = NeuralNet(input_size, hidden_size, num_classes).to(device) # step 1
2 criterion = nn.CrossEntropyLoss()                                   # step 2
3 optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate) # step 3
4 for epoch in range(num_epochs):
5     for i, (images, labels) in enumerate(train_loader):             # data: one batch
6         images = images.reshape(-1, 28*28).to(device)
7         labels = labels.to(device)
8         outputs = model(images)                                     # forward pass
9         loss = criterion(outputs, labels)
10        optimizer.zero_grad()                                       # clear old
   gradients
11    loss.backward()                                                  # backward pass
   (autograd)
  
```

12

`optimizer.step()`

update parameters

[PPT 第 26 页] 循环体与上一节“训练流程小结”的四个步骤一一对应。

⚠ zero_grad() 不能省

PyTorch 中梯度默认是累加的：每次 `backward()` 都会把新梯度加到 `.grad` 上。如果不清零，当前 batch 的梯度会和之前所有 batch 的梯度混在一起。另外，`nn.CrossEntropyLoss` 内部已经包含了 softmax，模型最后一层不要再加 softmax。

4 随机梯度下降可能存在的问题

4.1 问题一：所有参数共用一个学习率

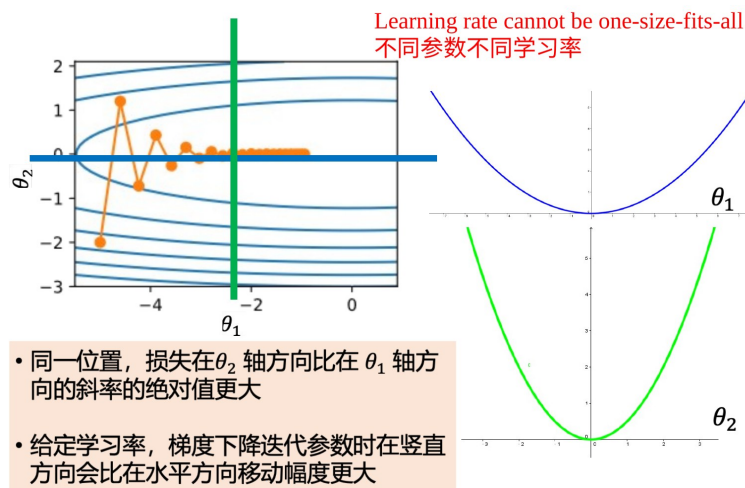


图 7 损失在 θ_2 方向陡峭、在 θ_1 方向平缓时，梯度下降的轨迹。[PPT 第 28 页]

- 同一位置，损失在 θ_2 轴方向的斜率绝对值比 θ_1 轴方向大得多；
- 给定学习率，梯度下降在竖直方向的移动幅度远大于水平方向：竖直方向来回震荡，水平方向进展缓慢；
- 学习率取大，陡峭方向发散；取小，平缓方向几乎不动。

结论：Learning rate cannot be one-size-fits-all——不同的参数应当使用不同的学习率。

4.2 问题二：梯度很小的地方

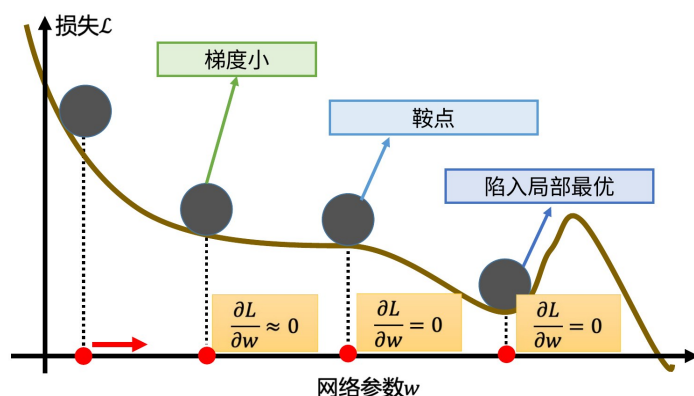


图 8 梯度下降可能停滞的三种位置。〔PPT 第 29 页〕

位置	梯度	说明
平坦区 (plateau)	$\partial L / \partial w \approx 0$	更新量正比于梯度，前进极慢
鞍点 (saddle point)	$\partial L / \partial w = 0$	某些方向是极小、另一些方向是极大；高维空间中鞍点远比局部极小值常见
局部最优	$\partial L / \partial w = 0$	四周都更高，单靠梯度无法离开

SGD 的梯度噪声在一定程度上有助于离开鞍点和较差的局部极小值，但更系统的办法是改进优化算法。

4.3 改进的优化算法（补充）

课件第 30 页用动画对比了 SGD、Momentum、NAG、Adagrad、Adadelata、RMSprop 在鞍点附近的表现，第 26 页的 PyTorch 代码使用了 Adam。这里补充它们的基本思想，记 $\mathbf{g}_t = \nabla \mathcal{L}(\boldsymbol{\theta}_{t-1})$ ，运算均为逐元素。

算法	更新规则	思想
Momentum	$v_t = \rho v_{t-1} + g_t, \quad \theta_t = \theta_{t-1} - \alpha v_t$	累积历史梯度形成“惯性”：震荡方向正负相消，一致方向不断加速；也能冲过平坦区和鞍点。针对 问题二
NAG	同上，但梯度在“前瞻位置” $\theta_{t-1} - \alpha \rho v_{t-1}$ 处计算	先按惯性走一步再看梯度，修正更及时
AdaGrad	$s_t = s_{t-1} + g_t^2, \quad \theta_t = \theta_{t-1} - \frac{\alpha}{\sqrt{s_t} + \epsilon} g_t$	每个参数有自己的学习率：历史梯度大的方向步子小，梯度小的方向步子大。针对 问题一 ；缺点是 s_t 只增不减，学习率过早衰减到 0
RMSProp	$s_t = \gamma s_{t-1} + (1 - \gamma) g_t^2$ ，其余同 AdaGrad	把累加改为指数滑动平均，解决 AdaGrad 的过早衰减 (Adadelta 思路类似)
Adam	$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad s_t = \beta_2 s_{t-1} + (1 - \beta_2) g_t^2, \quad \theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{s}_t} + \epsilon}$	Momentum + RMSProp，并对 m, s 做偏差修正 ($\hat{m}_t = m_t / (1 - \beta_1^t)$ 等)。目前最常用的默认选择

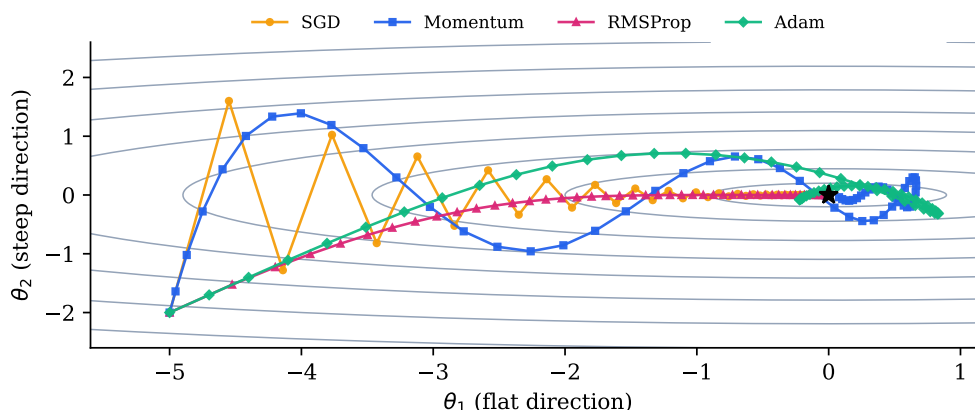


图9 在一个 θ_2 方向陡、 θ_1 方向平的二次函数上，四种算法 60 步的轨迹（笔者绘制）。SGD 在陡峭方向震荡、在平缓方向爬行；自适应学习率的方法（RMSProp、Adam）几乎沿直线走向最优解。

5 神经网络训练优化要点与技巧

5.1 样本随机性

- 每次迭代随机选择 B 个样本，这里的**随机性非常重要**：它保证小批量梯度是批量梯度的无偏估计。若数据按类别排好序，一个 batch 里全是同一类样本，梯度会严重偏向该类。[PPT 第 32 页]
- 但在大规模数据情况下，真正的随机采样很难满足（对磁盘随机访问的代价很高）。
- 实践中通常将样本顺序**打乱一次**，然后按这个顺序存储起来，之后顺序读取。虽然偏离了真实的随机采样，但不会有严重的有害影响。

```

1 train_loader = torch.utils.data.DataLoader(dataset=train_dataset,
2                                           batch_size=batch_size,
3                                           shuffle=True)
4 test_loader  = torch.utils.data.DataLoader(dataset=test_dataset,
5                                           batch_size=batch_size,
6                                           shuffle=False)

```

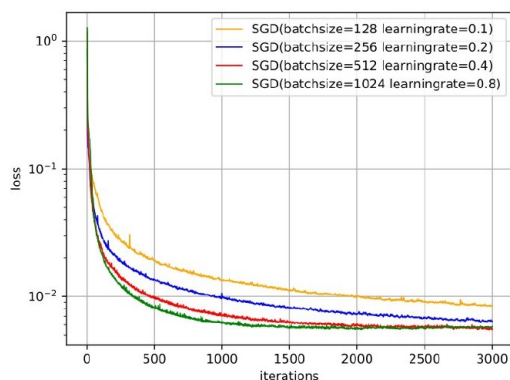
训练集 `shuffle=True`（每个 epoch 重新打乱）；测试集不需要打乱，因为测试时不更新参数，顺序不影响结果。

5.2 批量大小

✓ 核心结论

批量大小**不影响随机梯度的期望，但会影响随机梯度的方差**： $\text{Var}[\nabla \mathcal{L}_B] \propto 1/B$ 。[PPT 第 33 页]

- 批量越大，随机梯度的方差越小，引入的噪声越小，训练越稳定，因此**可以设置较大的学习率**；
- 批量较小时，需要设置较小的学习率，否则模型会不收敛；
- 批量大小一般取 2 的幂（32、64、128……），便于硬件对齐和并行。



(a) 按 Iteration 的损失变化

图中四组设置为 (批量, 学习率) = (128, 0.1)、(256, 0.2)、(512, 0.4)、(1024, 0.8), 即批量翻倍时学习率也翻倍——这就是常用的**线性缩放规则** (补充)。按迭代次数看, 批量越大下降越快; 但每次迭代的计算量也成倍增加, 按 epoch 看则未必。

图 10 小批量梯度下降中, 每次选取的样本数量对损失下降的影响。

5.3 学习率与学习率衰减

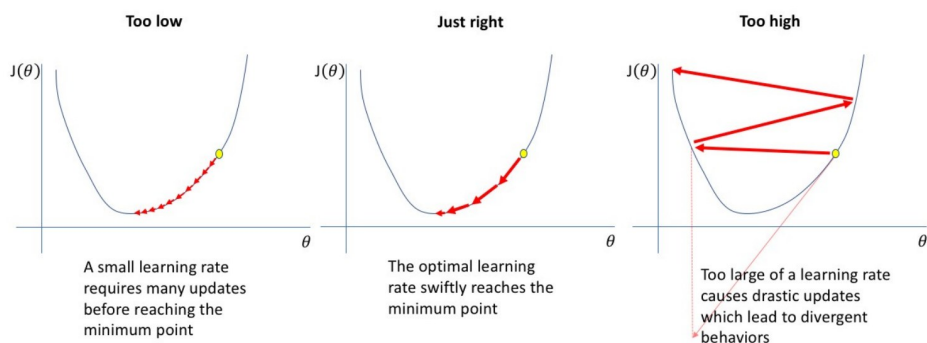
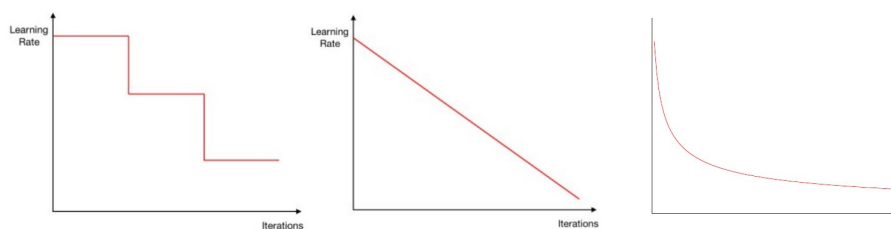


图 11 学习率太小: 需要很多次更新; 合适: 快速到达最低点; 太大: 更新过猛, 来回震荡甚至发散。[PPT 第 34 页]

固定的学习率很难兼顾全程, 于是采用**学习率衰减** [PPT 第 35 页]: 学习率一开始要保持大些, 以保证收敛速度; 收敛到最优点附近时要小些, 以避免来回振荡。

方式	形式 (补充)	特点
梯级衰减 (step decay)	$\alpha_t = \alpha_0 \gamma^{\lfloor t/T \rfloor}, 0 < \gamma < 1$	每隔 T 次迭代 (或若干 epoch) 乘一个系数, 如 0.1
线性衰减 (linear decay)	$\alpha_t = \alpha_0 (1 - t/T_{\max})$	线性降到 0
1/t 衰减	$\alpha_t = \alpha_0 / (1 + \beta t)$	前期下降快, 后期下降慢



梯级衰减 (step decay) 线性衰减 (Linear Decay) 1/t 衰减 (1/t decay)

图 12 三种学习率衰减方式。[PPT 第 36 页]

5.4 正则化

以上技巧关注“优化”（把训练损失降下去）；从这里开始关注“泛化”（防止过拟合）。

新的优化目标：不仅最小化损失，而且要让权重尽可能小。〔PPT 第 37 页〕

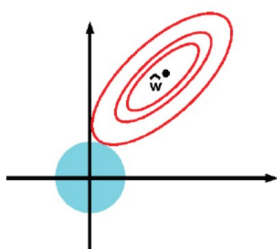
$$\mathcal{L}'(\theta) = \underbrace{\mathcal{L}(\theta)}_{\text{原损失: 平方损失、交叉熵等}} + \underbrace{\frac{1}{2}\lambda\|\theta\|_2^2}_{\text{正则化项}}, \quad \theta = \{w_1, w_2, \dots\}. \quad (5.1)$$

- **L2 正则：** $\|\theta\|_2^2 = w_1^2 + w_2^2 + \dots$ ； **L1 正则：** $\|\theta\|_1 = |w_1| + |w_2| + \dots$ 。
- 正则化**一般不考虑偏置**：偏置只平移函数，不影响函数的平滑程度（对输入变化的敏感度）。
- 为什么“权重小”有助于泛化：权重小，输出对输入的微小变化不敏感，函数更平滑，不容易去拟合训练数据中的噪声。

L2 正则 = 权重衰减。〔PPT 第 38 页〕 梯度为 $\nabla\mathcal{L}' = \nabla\mathcal{L} + \lambda\theta$ ，代入梯度下降：

$$\theta^t \leftarrow \theta^{t-1} - \eta\nabla\mathcal{L}' = \theta^{t-1} - \eta\nabla\mathcal{L} - \eta\lambda\theta^{t-1} = \underbrace{(1 - \eta\lambda)}_{<1} \theta^{t-1} - \eta\nabla\mathcal{L}. \quad (5.2)$$

由于 $\eta < 1, \lambda < 1$ ，每次更新都先把参数乘上一个略小于 1 的系数，参数越来越小；但由于 $-\eta\nabla\mathcal{L}$ 项的存在，参数不会变为 0。因此 L2 正则也称**权重衰减**（weight decay）。



几何解释：红色椭圆是原损失的等高线，中心 $\hat{w}$ 是不加正则时的最优解；蓝色圆是 L2 范数的约束区域。加正则后的解是两者之间的折中，被从 $\hat{w}$ 拉向原点。

L1 与 L2 的区别（补充）：L1 正则的梯度是 $\lambda \text{sign}(\theta)$ ，每次减去一个固定大小的量，与参数自身大小无关，因此会把许多参数精确地压到 0，得到**稀疏解**；L2 按比例缩小，参数变小但一般不为 0。

图 13 L2 正则的几何解释。

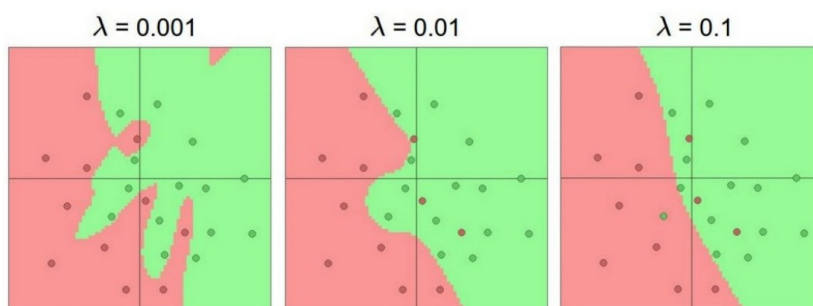


图 14 不同正则项权重 λ 对分类边界的影响 (<http://playground.tensorflow.org/>)。〔PPT 第 39 页〕

$\lambda = 0.001$ ：边界曲折，迁就每一个训练点，过拟合； $\lambda = 0.01$ ：边界变得平滑； $\lambda = 0.1$ ：边界近乎直线，模型过于简单，趋于欠拟合。 λ 是需要在验证集上选择的超参数。

5.5 提前终止 (Early Stop)

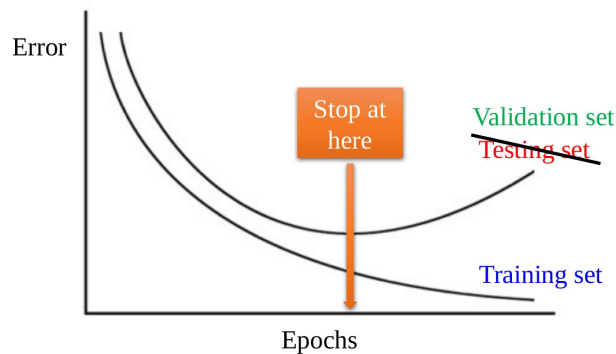


图 15 训练误差持续下降，验证误差先降后升；在验证误差最低处停止。〔PPT 第 40 页〕

- 验证集上准确率下降（损失上升）的时候停止训练；
- 更稳妥的做法：训练足够长时间，**保存在验证集上最优的模型**（验证曲线有噪声，一次上升未必是真的拐点）。

⚠ 必须是验证集，不能是测试集

课件图中特意把“Testing set”划掉、改成了“Validation set”。如果用测试集来决定何时停止，测试集就参与了模型选择，在它上面报告的性能会偏乐观，不再是对泛化能力的公正估计。

5.6 Dropout

5.6.1 训练时

取得一个 batch 后，以概率 p 随机把一些神经元的输出设置为 0。〔PPT 第 41–42 页〕

- 被置 0 的神经元连同它的所有连接相当于被删除，**网络结构发生了变化**，得到一个更“瘦”的新网络；
- 用新网络在该 batch 上计算梯度，在新网络上进行参数更新（被丢弃的神经元本轮不更新）；
- **每次取新 batch 时，都需要重新随机采样**哪些神经元被置为 0。

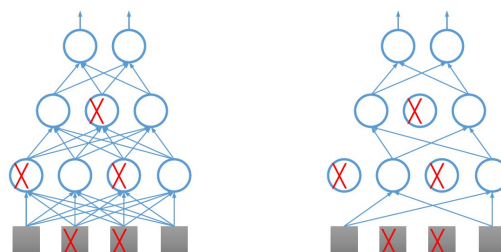


图 16 左：随机选中要丢弃的神经元；右：删除后得到的“瘦”网络。

5.6.2 测试时

测试时没有 dropout，所有神经元都处于激活状态。但这样每个神经元收到的总输入会比训练时大，因此需要**缩放**，使每个神经元测试时的输出等于训练时的**期望**输出。〔PPT 第 44 页〕

权重比例推断规则 (weight scaling inference rule)

如果训练时 dropout 的（丢弃）概率为 p ，那么测试时所有的权重要乘以 $1 - p$ （即保留概率）。

直观解释

丢弃概率为 0.5，某神经元有 4 个输入，训练时平均只有 2 个有效，净输入为 z 。测试时 4 个输入全部有效，若直接用训练后的权重，则 $z' \approx 2z$ ；把权重乘以 $1 - p = 0.5$ 之后， $z' \approx z$ ，与训练时一致。〔PPT 第 46 页〕

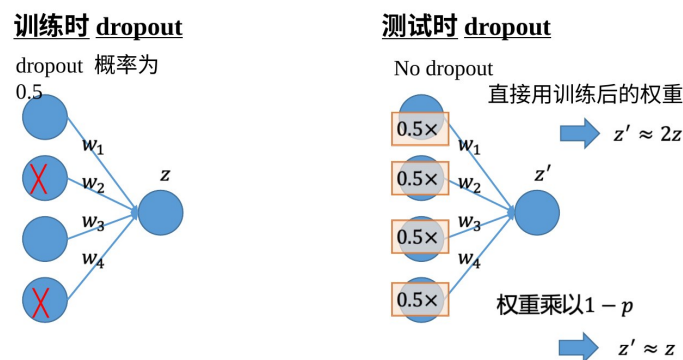


图 17 训练时 dropout 与测试时的权重缩放。

5.6.3 实现

```

1  p = 0.5 # probability of KEEPING a unit active. higher = less dropout
2  def train_step(X):
3      H1 = np.maximum(0, np.dot(W1, X) + b1)
4      U1 = np.random.rand(*H1.shape) < p # first dropout mask
5      H1 *= U1 # drop!
6      H2 = np.maximum(0, np.dot(W2, H1) + b2)
7      U2 = np.random.rand(*H2.shape) < p # second dropout mask
8      H2 *= U2 # drop!
9      out = np.dot(W3, H2) + b3
10 def predict(X):
11     H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the
12         activations
13     H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the
14         activations
15     out = np.dot(W3, H2) + b3

```

[PPT 第 43, 45 页] 经验取值：输入单元的保留概率取 0.8，隐藏单元取 0.5。

⚠ 注意 p 的两种含义

课件第 41–42、44、46 页的文字中， p 是**丢弃概率**（测试时乘 $1 - p$ ）；而第 43、45 页的代码中， p 是**保留概率**（测试时乘 p ）。两者说的是同一件事：**测试时乘以保留概率**。查阅资料和调用框架时务必先确认含义——PyTorch 的 `nn.Dropout(p)` 中 p 是丢弃概率。

Inverted dropout（补充）：实际框架采用等价的另一种实现——训练时就把保留下来的激活值除以保留概率（`H1 *= U1 / p_keep`），测试时什么都不用做。这样推理代码与是否使用 dropout 无关。这也是为什么 PyTorch 中必须用 `model.train()` / `model.eval()` 切换模式。

5.6.4 动机：为什么 Dropout 有效

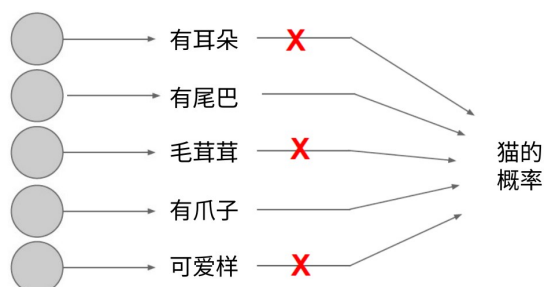


图 18 判断“是不是猫”的五个特征，随机丢掉其中一些。[PPT 第 47 页]

1. **强制网络学习冗余的特征表示**。任何一个特征都随时可能被丢掉，网络不能把判断押在某一个特征上；即使“有耳朵”“毛茸茸”“可爱样”被丢弃，仅凭“有尾巴”“有爪子”也应该能给出合理的判断。
2. **防止特征的 co-adaptation**。co-adaptation 指一个特征检测器只有在某些其他特定的特征检测器存在时才能发挥作用的情况；这种脆弱的“互相依赖”往往是在拟合训练集的偶然性。Dropout 使任何神经元都不能依赖特定的同伴。

人脸识别的例子 [PPT 第 48 页]：设模型学到了一个通过“鼻子”检测人脸的隐藏单元 h_i 。丢失 h_i 相当于擦除了“图像中有鼻子”的信息，模型必须学习另一种 h_i ：要么是鼻子存在的冗余编码，要么是像嘴这样的脸部的另一特征。

集成学习的视角（补充）：含 n 个可丢弃神经元的网络有 2^n 个子网络，它们共享参数。Dropout 每个 batch 训练其中一个；测试时的权重缩放，近似于对所有子网络的预测取平均。

5.6.5 DropConnect

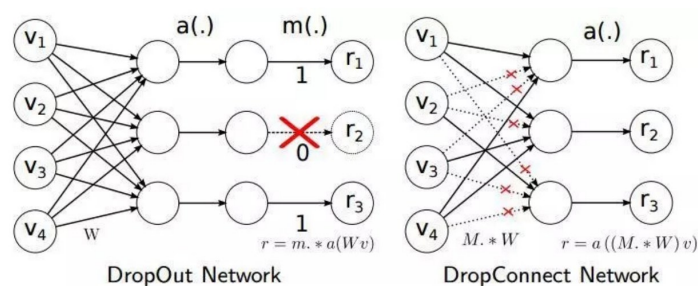


图 19 Dropout 丢弃的是神经元的输出；DropConnect 丢弃的是单条连接。〔PPT 第 49 页〕

- **Dropout**: 随机地将隐层节点的**输出**变成 0, $r = m \odot a(Wv)$ ——相当于删掉权重矩阵的一整行；
- **DropConnect**: 将节点的每个输入**权值**以 $1-p$ 的概率变成 0 (p 为保留概率), $r = a((M \odot W)v)$ ——逐个元素地删, 粒度更细, Dropout 是它的特例。

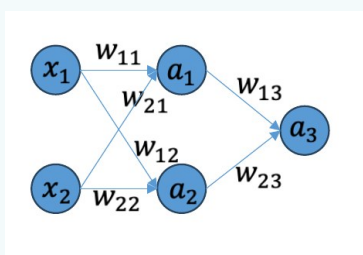
6 参数初始化

6.1 参数不能全部初始化为 0

对称权重问题

前馈神经网络中, 若同一层的权重初始化为相同的值, 则: 前向传播中每一层的不同神经元激活值相同; 反向传播中同一层各个权重参数的梯度值相同。这使得隐层神经元没有区分性, 神经网络无法有效学习数据中的特征。〔PPT 第 51 页〕

2-2-1 网络, 全零初始化



前向: $a_1 = f(w_{11}x_1 + w_{21}x_2 + b_1) = f(0)$, $a_2 = f(0)$, 两个隐层神经元输出相同。

第一次更新: 因 $a_1 = a_2$, 有 $\frac{\partial L}{\partial w_{13}} = \frac{\partial L}{\partial w_{23}}$, 更新后仍有 $w_{13} = w_{23}$; 传回隐层的误差要乘以 $w_{13} = w_{23} = 0$, 故 $\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial w_{12}} = \frac{\partial L}{\partial w_{21}} = \frac{\partial L}{\partial w_{22}} = 0$ 。

随后的更新: $w_{13} = w_{23}$ 且 $a_1 = a_2$, 两个隐层神经元收到的误差始终相同, 由对称性一直保持 $a_1 = a_2$ 。

结果: 无论隐层有多少个神经元, 它们的行为都完全相同, 网络的有效宽度为 1。不只是全零——任何“全部相同”的初始化都有此问题, 必须用**随机性**来打破对称。

6.2 初始化方法

- **预训练初始化**: 用在大规模数据上训练好的模型参数作为初值, 再在目标任务上微调;
- **随机初始化**: 本节及下一节的重点;
- **固定值初始化**: 例如偏置通常用 0 初始化——对称性已由随机的权重打破, 偏置取 0 没有问题。

6.3 基于固定方差的随机初始化

- **高斯分布初始化**: 参数从固定均值 (如 0) 和固定方差 (如 0.01) 的高斯分布中随机采样,
 $W = \text{sigma} * \text{np.random.randn}(\text{fan_in}, \text{fan_out});$
- **均匀分布初始化**: 参数在区间 $[-r, r]$ 内均匀采样, $W = 2*r*\text{np.random.rand}(\text{fan_in}, \text{fan_out}) - r$ 。

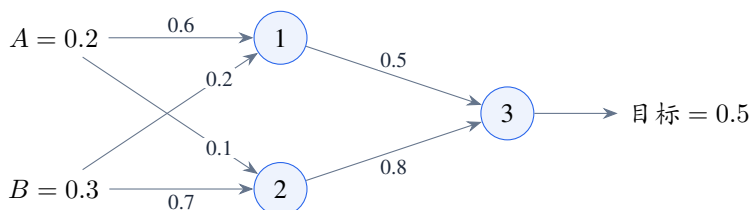
$[-r, r]$ 上均匀分布的方差为 $(2r)^2/12 = r^2/3$, 所以若方差为 σ^2 , 则 $r = \sqrt{3\sigma^2}$ 。 [PPT 第 52 页]

💡 预告

“固定方差”到底取多大? 太小, 信号逐层衰减; 太大, 激活函数饱和。合适的方差应当由每层的宽度决定——这就是下一节 (3.3) 的 Xavier 初始化与 He 初始化。

7 课后习题与参考解答

题目 [PPT 第 53 页]: 在如下神经网络中, 假设激活函数为 ReLU, 用平方损失 $\frac{1}{2}(y - \hat{y})^2$ 计算误差。请用 BP 算法更新一次所有参数 (学习率为 1), 给出更新后的参数值, 并计算给定输入 $x = (0.2, 0.3)$ 时初始时和更新后的输出值, 检查参数更新是否降低了平方损失值。



参考解答 (笔者给出, 仅供自查; 权重与神经元的对应关系按课件第 18 页例题的画法读取: $w_{A1} = 0.6$, $w_{B1} = 0.2$, $w_{A2} = 0.1$, $w_{B2} = 0.7$, $w_{13} = 0.5$, $w_{23} = 0.8$)。采用 $e = \partial \mathcal{L} / \partial z$ 的约定, 更新为 $w \leftarrow w - e \cdot a$ 。

前向: $z_1 = 0.2 \times 0.6 + 0.3 \times 0.2 = 0.18$, $z_2 = 0.2 \times 0.1 + 0.3 \times 0.7 = 0.23$, 均为正, ReLU 后 $a_1 = 0.18$, $a_2 = 0.23$; $z_3 = 0.5 \times 0.18 + 0.8 \times 0.23 = 0.274$, $\hat{y} = 0.274$ 。初始损失 $\frac{1}{2}(0.5 - 0.274)^2 = 0.02554$ 。

后向: 所有净输入均为正, ReLU 的导数都是 1。

$$e_3 = (\hat{y} - y) \cdot 1 = -0.226, \quad e_1 = e_3 w_{13} \cdot 1 = -0.113, \quad e_2 = e_3 w_{23} \cdot 1 = -0.1808.$$

更新:

参数	计算	新值
w_{13}	$0.5 - (-0.226) \times 0.18$	0.54068
w_{23}	$0.8 - (-0.226) \times 0.23$	0.85198
w_{A1}	$0.6 - (-0.113) \times 0.2$	0.6226
w_{B1}	$0.2 - (-0.113) \times 0.3$	0.2339
w_{A2}	$0.1 - (-0.1808) \times 0.2$	0.13616
w_{B2}	$0.7 - (-0.1808) \times 0.3$	0.75424

验证： $z_1 = 0.19469$, $z_2 = 0.25350$, $\hat{y} = 0.54068 \times 0.19469 + 0.85198 \times 0.25350 = 0.32125$ 。损失 $\frac{1}{2}(0.5 - 0.32125)^2 = 0.01598$, **从 0.02554 降到 0.01598**, 输出从 0.274 向目标 0.5 靠近, 参数更新降低了平方损失。 ■

8 课件勘误与补充说明

页码	说明
5	更新公式 $\theta^{(l+1)} \leftarrow \theta^{(l)} - \alpha \partial \mathcal{L} / \partial \theta^{(l)}$ 中的上标 (l) 表示 迭代次数 , 与后文表示“层”的上标 (l) 含义不同; 第 6 页改用了 t 。
9	标题为“终止条件”; 所示算法为每次取单个样本 ($B = 1$) 的 SGD, 终止条件是验证集错误率不再下降。
13	左侧下半部分的“ $f_2(\cdot)$: 隐层激活函数”应为 $f_1(\cdot)$; “ $b_h^{(1)}$: 输出层神经元的偏置”应为 隐层 神经元的偏置。
14	训练样本为数字“4”, 标记下标从 d_0 开始, one-hot 中为 1 的应是 d_4 , 图中写成了 $d_3 = 1$ 。
15	前向示意中第二个箭头下方的 $f_1(z_j^{(2)})$ 应为 $f_2(z_j^{(2)})$; 该页公式省略了偏置。
16-17	红色公式把 $e_j^{(2)}$ 写作 $\frac{\partial \mathcal{L}}{\partial y_j} \frac{\partial y_j}{\partial z_j^{(2)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(2)}}$, 而同页又有 $e_j^{(2)} = (d_j - y_j)f_2'(z_j^{(2)})$, 两者相差一个负号。按后者及最终的更新式, 应为 $e_j^{(2)} = -\partial \mathcal{L} / \partial z_j^{(2)}$ (第 17 页同理)。最终的 Δw 公式是正确的。
19-21	此处误差项定义为 $e^{(l)} = +\partial \mathcal{L} / \partial z^{(l)}$, 与第 16-18 页相差负号; 递推式按行向量书写为 $e^{(l+1)} \mathbf{W}^{(l+1)}$ 。
25	Keras 代码为 1.x 旧版 API (output_dim, nb_epoch)。
37	“正则化项目”应为“正则化项”。
41-46	文字中的 p 为丢弃概率, 代码中的 p 为保留概率 (第 45 页已有标注), 含义不统一; 第 44 页“神经员”应为“神经元”。
49	“以 $1 - p$ 的概率变成 0”中的 p 为保留概率 (与 DropConnect 原论文一致)。
52	均匀分布的代码 $W = r * \text{np.random.rand}(\dots)$ 采样的是 $[0, r]$, 应为 $2*r*\text{np.random.rand}(\dots) - r$ 才是 $[-r, r]$ (3.3 节课件第 40 页的写法是正确的)。

9 自测问题

1. 梯度下降在什么条件下能保证损失单调不增? 随机梯度下降呢?
2. 为什么小批量梯度可以代替全量梯度? 批量大小影响它的什么性质?
3. 训练时的显存开销由哪几部分构成? 哪些与模型大小有关, 哪些与批量大小有关?
4. 反向传播为什么要“先前向、再后向”? 哪些量被存储复用了?

5. 默写后向传播的三条核心公式，并检查每一条的维度。
6. 隐层神经元的误差项为什么要对后一层的所有神经元求和？
7. PyTorch 训练循环中 `zero_grad()`、`backward()`、`step()` 各自对应训练流程的哪一步？
8. “不同参数需要不同学习率”和“梯度为 0 处停滞”这两个问题，分别由哪类优化算法应对？
9. 批量增大时，学习率应如何调整？为什么？
10. 推导：为什么 L2 正则等价于权重衰减？为什么参数不会衰减到 0？
11. 提前终止为什么必须使用验证集，而不是测试集？
12. Dropout 在测试时为什么要缩放？若丢弃概率为 0.3，测试时权重应乘以多少？
13. 什么是 co-adaptation？Dropout 如何防止它？
14. 为什么不能把所有权重初始化为同一个常数？偏置可以初始化为 0 吗？